

1. Pruebas data-driven y conexión de cobertura con `pytest-cov`

1.1. Objetivo de la implementación

Para reducir la duplicación de código y ejecutar un mismo flujo con distintos datos de entrada, se diseñó una prueba *data-driven* para el módulo de autenticación de Cal.com. La implementación utiliza el decorador `@pytest.mark.parametrize` y obtiene los datos desde un archivo CSV externo.

La separación entre la lógica de prueba y los datos permite ampliar los escenarios sin crear una función diferente para cada combinación. Este enfoque también mejora la mantenibilidad de la suite y facilita la identificación de fallas específicas.

La cobertura se integró mediante `pytest-cov`, con salida en terminal, reporte HTML y archivo XML compatible con SonarQube.

1.2. Diseño de los casos de prueba

La prueba valida el inicio de sesión de Cal.com con datos válidos, inválidos y de frontera. Los escenarios definidos se muestran en la Tabla 1.

Tabla 1: Casos data-driven para el inicio de sesión de Cal.com

ID	Datos de entrada	Resultado esperado
DD-01	Correo y contraseña válidos	El usuario inicia sesión y abandona la ruta <code>/auth/login</code> .
DD-02	Correo sin formato válido	El navegador muestra una validación del campo de correo.
DD-03	Correo vacío	El formulario marca el campo de correo como obligatorio.
DD-04	Contraseña vacía	El formulario marca el campo de contraseña como obligatorio.
DD-05	Contraseña incorrecta	La aplicación muestra un error de autenticación y no inicia sesión.

Las credenciales válidas se obtienen mediante las variables de entorno `CALCOM_TEST_EMAIL` y `CALCOM_TEST_PASSWORD`. Con ello, el repositorio puede conservar los casos de prueba sin exponer información sensible.

1.3. Archivo de datos CSV

Los datos de entrada se almacenaron en el archivo `tests/data/login_cases.csv`. Cada fila representa una ejecución independiente de la misma prueba.

```
1 id,email,password,expected,description
2 DD-01,ENV:CALCOM_TEST_EMAIL,ENV:CALCOM_TEST_PASSWORD,success,Credenciales validas
3 DD-02,correo-sin-arroba>Password123!,validation,Formato de correo invalido
4 DD-03,,Password123!,validation,Correo vacio
5 DD-04,usuario@example.com,,validation,Contrasena vacia
6 DD-05,usuario@example.com,ContrasenaIncorrecta123!,auth_error,Contrasena
  incorrecta
```

Listing 1: Archivo CSV con los casos de prueba data-driven

El prefijo `ENV:` identifica los valores que se recuperan desde variables de entorno. Esta estructura mantiene separados los datos sensibles y los datos versionados.

1.4. Implementación de la prueba parametrizada

La función `load_login_cases()` abre el archivo CSV, transforma cada fila en un diccionario y sustituye los valores con prefijo `ENV:` por el contenido de las variables de entorno correspondientes.

```
1 from __future__ import annotations
2
3 import csv
4 import os
5 from pathlib import Path
6
7 import pytest
8
9 from pages.login_page import LoginPage
10
11 DATA_FILE = Path(__file__).parent / "data" / "login_cases.csv"
```

```

12
13
14 def _resolve_value(value: str) -> str:
15     if value.startswith("ENV:"):
16         return os.getenv(value.removeprefix("ENV:"), "")
17     return value
18
19
20 def load_login_cases() -> list[dict[str, str]]:
21     with DATA_FILE.open(encoding="utf-8", newline="") as file:
22         cases = list(csv.DictReader(file))
23
24     for case in cases:
25         case["email"] = _resolve_value(case["email"])
26         case["password"] = _resolve_value(case["password"])
27     return cases
28
29
30 @pytest.mark.e2e
31 @pytest.mark.parametrize(
32     "case",
33     load_login_cases(),
34     ids=lambda case: case["id"],
35 )
36 def test_login_data_driven(
37     driver,
38     base_url: str,
39     case: dict[str, str],
40 ) -> None:
41     if case["id"] == "DD-01" and (
42         not case["email"] or not case["password"]
43     ):
44         pytest.skip(
45             "Credenciales de prueba no disponibles "
46             "en las variables de entorno."
47         )
48
49     login_page = LoginPage(driver, base_url)

```

```

50 login_page.open()
51 login_page.submit_credentials(
52     case["email"],
53     case["password"],
54 )
55
56 result = login_page.wait_for_result()
57
58 assert result.status == case["expected"], (
59     f"Caso {case['id']} ({case['description']}): "
60     f"se esperaba '{case['expected']}', "
61     f"pero se obtuvo '{result.status}'. "
62     f"Detalle: {result.message}"
63 )

```

Listing 2: Prueba data-driven con `@pytest.mark.parametrize`

Aunque existe una sola función, pytest registra cada fila del archivo CSV como una ejecución separada. Los casos DD-01, DD-02, DD-03, DD-04 y DD-05 aparecen individualmente en la terminal y en el reporte HTML.

El argumento `ids=lambda case: case["id"]` asigna a cada ejecución el identificador definido en el CSV. Esto mejora la lectura de los resultados y permite localizar con rapidez el conjunto de datos relacionado con una falla.

```

tests/test_login_data_driven.py::test_login_data_driven[DD-01] PASSED [ 20%]
tests/test_login_data_driven.py::test_login_data_driven[DD-02] PASSED [ 40%]
tests/test_login_data_driven.py::test_login_data_driven[DD-03] PASSED [ 60%]
tests/test_login_data_driven.py::test_login_data_driven[DD-04] PASSED [ 80%]
tests/test_login_data_driven.py::test_login_data_driven[DD-05] PASSED [100%]

```

Figura 1: Salida de pytest con la ejecución individual de los casos DD-01 a DD-05.

1.5. Integración con Page Object Model

La prueba se apoya en el Page Object `LoginPage`. Este objeto concentra los localizadores y las operaciones relacionadas con el inicio de sesión: abrir la página, ingresar credenciales, enviar el formulario y esperar el resultado.

```

1 from dataclasses import dataclass
2
3 from selenium.common.exceptions import TimeoutException

```

```

4 from selenium.webdriver.common.by import By
5 from selenium.webdriver.support import expected_conditions as EC
6 from selenium.webdriver.support.ui import WebDriverWait
7
8
9 @dataclass(frozen=True)
10 class LoginResult:
11     status: str
12     message: str = ""
13
14
15 class LoginPage:
16     EMAIL = (
17         By.CSS_SELECTOR,
18         'input[name="email"], input[type="email"]',
19     )
20     PASSWORD = (
21         By.CSS_SELECTOR,
22         'input[name="password"], input[type="password"]',
23     )
24     SUBMIT = (By.CSS_SELECTOR, 'button[type="submit"]')
25     ERROR = (
26         By.CSS_SELECTOR,
27         '[role="alert"], [data-testid="toast-error"], '
28         '[data-testid!="error"]',
29     )
30
31     def __init__(self, driver, base_url: str, timeout: int = 12):
32         self.driver = driver
33         self.base_url = base_url.rstrip("/")
34         self.wait = WebDriverWait(driver, timeout)
35
36     def open(self) -> None:
37         self.driver.get(f"{self.base_url}/auth/login")
38         self.wait.until(
39             EC.visibility_of_element_located(self.EMAIL)
40         )
41

```

```

42     def submit_credentials(self, email: str, password: str) -> None:
43         email_input = self.wait.until(
44             EC.visibility_of_element_located(self.EMAIL)
45         )
46         email_input.clear()
47         email_input.send_keys(email)
48
49         password_fields = self.driver.find_elements(*self.PASSWORD)
50         if not password_fields:
51             self.wait.until(
52                 EC.element_to_be_clickable(self.SUBMIT)
53             ).click()
54             password_input = self.wait.until(
55                 EC.visibility_of_element_located(self.PASSWORD)
56             )
57         else:
58             password_input = password_fields[0]
59
60         password_input.clear()
61         password_input.send_keys(password)
62         self.wait.until(
63             EC.element_to_be_clickable(self.SUBMIT)
64         ).click()

```

Listing 3: Fragmento principal del Page Object LoginPage

La sincronización con la interfaz se realizó mediante esperas explícitas con `WebDriverWait`. El uso de condiciones explícitas evita depender de pausas fijas y reduce la inestabilidad de las pruebas.

Los localizadores consideran atributos comunes del formulario de autenticación, como `name`, `type`, `role` y `data-testid`.

1.6. Configuración de pytest y pytest-cov

La suite utiliza Selenium, pytest, pytest-cov y pytest-html. Las dependencias se instalaron con el siguiente comando:

```

1 pip install selenium pytest pytest-cov pytest-html

```

Listing 4: Instalación de dependencias para la suite

La configuración de pytest se almacenó en el archivo `pytest.ini`:

```
1 [pytest]
2 testpaths = tests
3 python_files = test_*.py
4
5 markers =
6     e2e: pruebas de extremo a extremo con navegador
7
8 addopts =
9     -v
10    --strict-markers
11    --html=reports/reporte_pytest.html
12    --self-contained-html
13    --cov=pages
14    --cov-branch
15    --cov-report=term-missing
16    --cov-report=xml:reports/coverage.xml
17    --cov-report=html:reports/htmlcov
```

Listing 5: Configuración del archivo `pytest.ini`

Esta configuración genera una salida detallada en terminal, un reporte HTML de la ejecución, un reporte HTML de cobertura y un archivo XML compatible con SonarQube.

La ejecución principal queda reducida al comando siguiente:

```
1 pytest
```

Listing 6: Ejecución mediante la configuración de `pytest.ini`

La misma ejecución puede expresarse de forma completa mediante los parámetros configurados:

```
1 pytest -v \
2     --cov=pages \
3     --cov-branch \
4     --cov-report=term-missing \
5     --cov-report=xml:reports/coverage.xml \
6     --cov-report=html:reports/htmlcov \
```

```
7  --html=reports/reporte_pytest.html \
8  --self-contained-html
```

Listing 7: Ejecución de pruebas y generación de reportes

1.7. Configuración del entorno de prueba

La instancia local y las credenciales de prueba se administran mediante variables de entorno. En PowerShell se utilizó la siguiente configuración:

```
1 $env:CALCOM_BASE_URL="http://localhost:3000"
2 $env:CALCOM_TEST_EMAIL="usuario_prueba@ejemplo.com"
3 $env:CALCOM_TEST_PASSWORD="ContrasenaDePrueba123!"
4 pytest
```

Listing 8: Variables de entorno en PowerShell

La configuración equivalente en Linux o macOS es la siguiente:

```
1 export CALCOM_BASE_URL="http://localhost:3000"
2 export CALCOM_TEST_EMAIL="usuario_prueba@ejemplo.com"
3 export CALCOM_TEST_PASSWORD="ContrasenaDePrueba123!"
4 pytest
```

Listing 9: Variables de entorno en Linux o macOS

1.8. Reportes generados

La ejecución genera la siguiente estructura de archivos:

```
1 reports/
2 |-- coverage.xml
3 |-- reporte_pytest.html
4 '-- htmlcov/
5     '-- index.html
```

Listing 10: Archivos generados por pytest

El archivo `reporte_pytest.html` contiene el resultado de cada caso de prueba. El archivo `htmlcov/index.html` presenta la cobertura por archivo y por línea. El archivo `coverage.xml` contiene la información de cobertura utilizada por SonarQube.

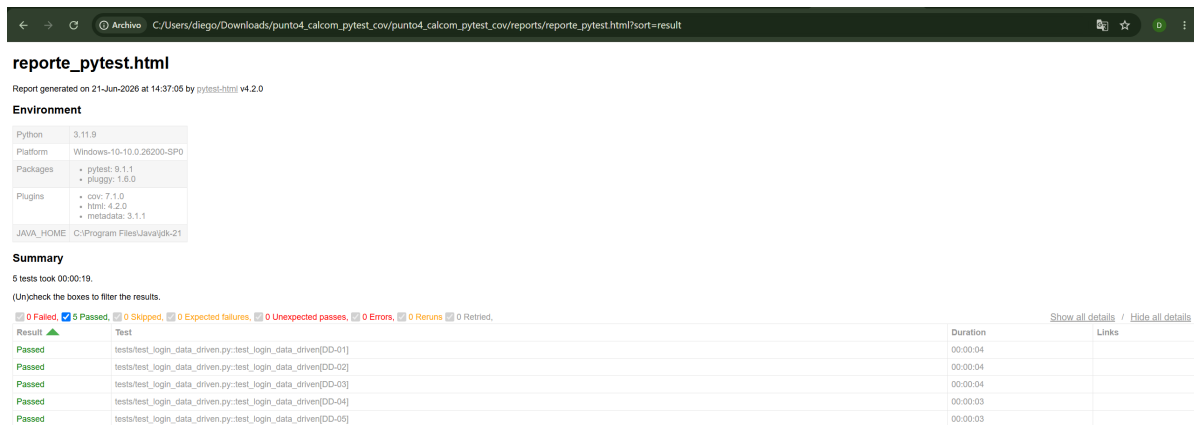


Figura 2: Reporte HTML generado por `pytest-html`.

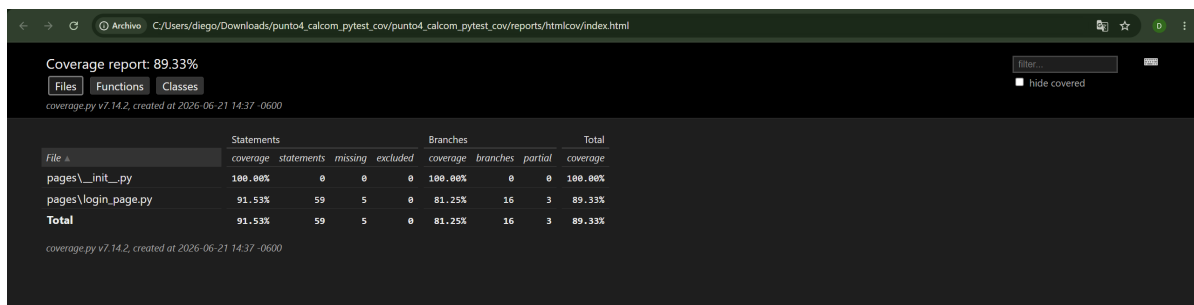


Figura 3: Reporte de cobertura por archivo y por línea generado por `pytest-cov`.

1.9. Conexión de la cobertura con SonarQube

La cobertura XML se incorporó al análisis mediante la propiedad `sonar.python.coverage.reportPaths` del archivo `sonar-project.properties`:

```
1 sonar.python.coverage.reportPaths=selenium_tests/reports/coverage.xml
```

Listing 11: Configuración de cobertura Python en SonarQube

Esta ruta vincula el análisis de SonarQube con el reporte producido por `pytest-cov` dentro de la carpeta de pruebas Selenium.

`pytest-cov` mide las líneas de código Python ejecutadas durante la suite. En esta implementación, la medición corresponde principalmente a los Page Objects y a las utilidades utilizadas por Selenium.

Cal.com está desarrollado principalmente con Next.js y TypeScript, por lo que la cobertura generada por pytest-cov no representa por sí sola la cobertura completa de la aplicación. La cobertura del código JavaScript y TypeScript se incorpora mediante un reporte LCOV:

```
1 sonar.javascript.lcov.reportPaths=coverage/lcov.info
```

Listing 12: Configuración de cobertura JavaScript y TypeScript en SonarQube

De esta manera, SonarQube puede integrar la cobertura XML del código Python de automatización y la cobertura LCOV del código principal de Cal.com sin confundir ambas mediciones.



Figura 4: Cobertura registrada en el tablero de SonarQube.

1.10. Interpretación de la implementación

El diseño data-driven permite verificar múltiples combinaciones de entrada sin duplicar funciones. Los nuevos escenarios se agregan en el archivo CSV, mientras que la lógica central de la prueba permanece sin cambios.

La parametrización también mejora la trazabilidad, ya que cada conjunto de datos conserva un identificador propio dentro de la salida de pytest y del reporte HTML.

El reporte `term-missing` señala las líneas de Python que no fueron ejecutadas. El reporte HTML facilita la revisión visual por archivo, mientras que el archivo `coverage.xml` permite trasladar la métrica a SonarQube.

La separación entre la cobertura de Python y la cobertura de TypeScript evita interpretar la ejecución de los Page Objects como si representara la totalidad del código de Cal.com.

1.11. Conclusión

La implementación integró una prueba data-driven mediante `@pytest.mark.parametrize`, datos externos en formato CSV y variables de entorno para proteger las credenciales. La estructura permite ejecutar casos válidos, inválidos y de frontera desde una sola función de prueba.

La integración con `pytest-cov` genera resultados en terminal, un reporte HTML y un archivo XML de cobertura. Este último se conecta con SonarQube mediante `sonar.python.coverage.reportPaths`.

El enfoque mejora la mantenibilidad, la trazabilidad y la integración de la suite con procesos posteriores de análisis de calidad e integración continua.

Referencias utilizadas en esta sección

Aniche, M. (2022). *Effective software testing: A developer's guide*. Manning.

García, B. (2022). *Hands-on Selenium WebDriver with Java*. O'Reilly Media.

pytest. (s. f.). *How to parametrize fixtures and test functions*. Documentación oficial de pytest.

pytest-cov. (s. f.). *Coverage reporting with pytest-cov*. Documentación oficial de pytest-cov.

SonarSource. (s. f.). *Python test coverage*. Documentación oficial de SonarQube.